



Sécurité et **A**dministration des **S**ystèmes **I**nformatiques

Sécurité des applications

Fabrice Legond-Aubry
Fabrice.Legond-Aubry@parisnanterre.fr



Plan de cours

Introduction

Attaques génériques de services

Attaques spécifiques de services

Backdoors / Rootkits / Trojans

Outils génériques de protection

Audit et check-list

Introduction



Des attaques variées

Quand un programme interagit avec son environnement

- Il est menacé
- Une attaque peut avoir lieu sur chaque point d'interaction
- Les accès disques, les paramètres d'exécution, les sockets, la mémoire sont des points d'interactions
- Selon les points d'interactions des attaques distantes et/ou locales sont possibles
- Les programmes « suid » sont des cibles de choix !

Les services externes/publiques sont très exposés !

Rappel: <http://www.ouah.org>, <http://www.phrack.org>

Les sites WEB dynamiques sont les plus exposés

- Attaques par des scripts php, CGI
- Attaques par injection ➔ Attaque Virus HTML
- Cross scripting



Des attaques variées

On constate des classes d'attaques sur les services. Elles sont « génériques »

- Le « buffer overflow » est la plus connue
- L'attaque par chaîne de format
- Valeurs hors normes sur les services [local et distant]
- Par virus, cheval troie, ...

Les attaques génériques sont une première étape vers le compromission de la machine. Elles permettent

- l'accès à la machine
- l'installation des modules noyaux, de backdoor, de rootkit
- l'utilisation de la machine à des fins malveillantes

D'autres services peuvent subir des compromissions spécifiques pour détourner leur utilisation

- DNS cache poisoning
- Bounce FTP server attack



Une journée ordinaire de M^r ROOT

Voici les avis publiés le 10 janvier 2006 :

- [cert-renater]** [AVIS CERTR : OpenBSD: Patch fixes suid /dev/fd access check]
- [cert-renater]** [AVIS CERTR : Mandriva Linux: Updated xpdf packages fix several
- [cert-renater]** [AVIS CERTR : RedHat: httpd security update]
- [cert-renater]** [AVIS CERTR : Debian: New smstools packages fix format string v
- [cert-renater]** [AVIS CERTR : Ubuntu: sudo vulnerability]
- [cert-renater]** [AVIS CERTR : Gentoo: VMware Workstation Vulnerability in NAT n
- [cert-renater]** [AVIS CERTR : SCO: LibXpm Integer Overflow Vulnerability]
- [cert-a]** [AVIS CERTA : Vulnérabilité dans ClamAV]
- [cert-a]** [AVIS CERTA : Vulnérabilité du module mod ssl dans Apache 2]
- [cert-a]** [AVIS CERTA : Multiples vulnérabilités dans postgresQL]
- [cert-a]** [AVIS CERTA : Vulnérabilité dans auth ldap pour Apache]



Les méthodes de protections

Des protections contre ces attaques existent

- Des outils de contrôle dynamiques
 - libsafe, lids, systrace, TCPwrapper
- Des outils d'isolation et de virtualisation
 - vmware, vserver, uml, chroot
- Des audits automatiques de code
 - Algorithmes d'analyse du code source
 - Vérification des « include » dans les scripts, échappement des chaînes de caractères
- Limiter services et contrôler les services
 - Eliminer les mots de passes en clair, PKI

Un système 100% sûr n'existant pas, il faut prévoir

- Des outils d'audit
 - Forensics Analysis (log, disque dur, ...)
 - Détection des Root kit
 - Vérification de l'intégrité des fichiers
- Des check-lists de sécurité à vérifier régulièrement
- Des pots de miels



Plan de cours

Introduction

Attaques génériques de services

Attaques spécifiques de services

Backdoors / Rootkits / Trojans

Outils génériques de protection

Audit et check-list

Attaques génériques de services



Attaques par execv

Attaque très simple basée sur un appel exec mal protégé

- Des programmeurs imprudents : Pas de chemin absolu !
- `execve (« ls », NULL, NULL)`

Il suffit de faire en sorte que le programme exécute son propre « ls »

- Un exec modifié c'est mieux !

Si le programme appartient à « root » et a le bit suid à 1, le « ls » sera exécuté avec les droits « root »

Exemple

```
echo « xterm & ls » > /tmp/ls
EXPORT PATH=/tmp:$PATH
./programme_mal_ecrit
```

Solution :

- Faire un `setuid`, `setgid` lors d'appel comme `system`, `exec`, ...
- Fixer des chemins absolus
- Purger l'environnement (PATH !!!)
- Eviter les programmes externes → il y a toujours un autre moyen



Attaques par débordement

- Terme anglais : « buffer overflow » ou « bof »
- Principes
 - Etre capable d'exécuter un code arbitraire à travers un autre programme
 - ✓ Le code arbitraire est souvent un « code SHELL »
 - ✓ Le programme doit être « suid » pour être intéressant
- But
 - Obtenir un accès maximum sur un système
 - ✓ « Root » sur linux/unix; « Administrator » sur XP
- Conséquences
 - Le code « shell » peut être utilisé pour modifier n'importe quoi sur la machine
 - ✓ Installer des traps, ouvrir un terminal « root »
 - ✓ Télécharger des rootkits et les exécuter



Attaques par débordement

- C'est un problème majeur
 - Il n'est pas ENCORE démodé !! Mais en perte légère de vitesse.
 - Des nombreux (tous ?) services ont déjà été affectés
 - ✓ HTTP (apache, iis), ftp (proftpd, ftpd)
 - ✓ imap (TOUS) , smtp (sendmail), syslog
 - De nombreuses commandes ont déjà été affectées
 - ✓ Tous les systèmes UNIX/linux/Windows
 - ✓ Eject, rsync, rdist, cvs, mount
 - Laisse peu de traces (arrêt du service)
- Les serveurs Web sont particulièrement exposés
 - De nombreux scripts CGI, pages php/asp/jsp
 - Ils sont mal conçus/pensés par des personnes non spécialistes
- **Essentiellement du à de mauvaises habitudes de programmation**
 - **Pourrait être facilement évité !**



Attaques par débordement - Solutions

- Une bonne programmation c'est :
 - Eviter les manipulations sans contrôle
 - ✓ **BANNIR strcpy(), get(), strcat()**
 - ✓ **ATTENTION AUX sprintf(), vsprintf(), scanf()**
 - ✓ Utilisation de strcpy() → strncpy()
 - ✓ Utilisation de gets() → fgets()
 - Utiliser des bibliothèques spéciales
 - ✓ **Vérification systématiques des bornes**
 - Utiliser des versions spéciales de malloc()
 - ✓ Vérification des allocations et des bornes
 - Utiliser des bibliothèques de debug pour vérifier le programme
 - ✓ Dbmalloc, checker, LeakTracer, ...
 - Purger l'environnement d'exécution !
- Modification du noyau
 - Création de nouvelles limitations sur le processus
 - Interdiction d'invocation de fork() et/ou d'exec()



Attaques par débordement

Concerne les programmes s'exécutant avec les droits « root »
(user/group) et avec le bit suid

- Exécuter par des utilisateurs normaux mais devant conserver les droits « root »

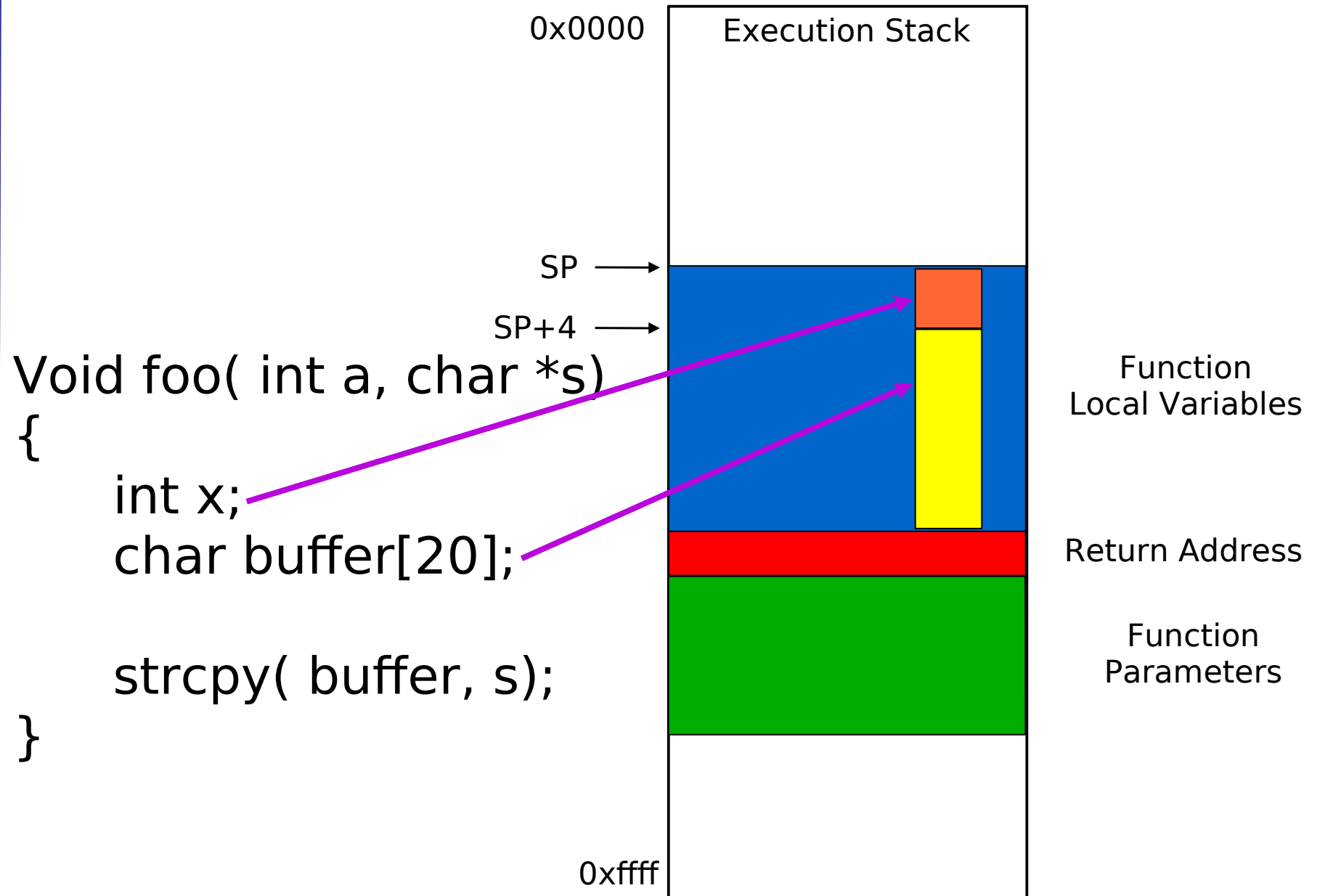
Sous MacOS X, on peut corrompre des programmes ayant comme propriétaire « admin » ou « root »

Prologue ➔ trouver ces programmes

```
find /usr/bin -perm /u+s -print0 | \  
    xargs -0 -i ls -ail "{}" > suid.lst  
echo "see in suid.lst for the list..."
```

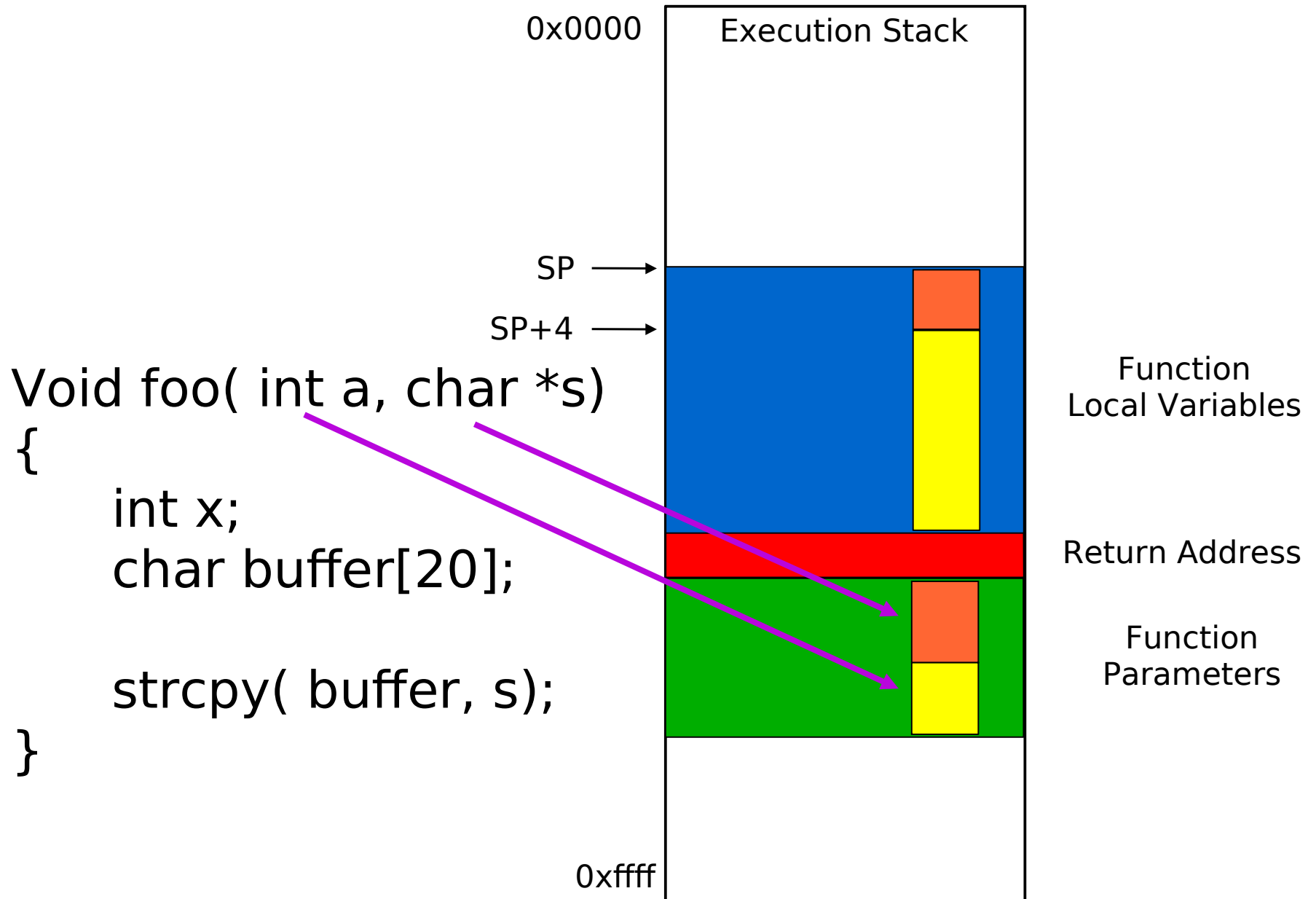


Attaques par débordement de Pile : principes





Attaques par débordement de Pile : principes





Attaques par débordement de Pile : principes

```
Void foo( int a, char  
*s)  
{  
    int x;  
    char buffer[20];  
  
    strcpy( buffer, s);  
}
```

But:

Transmettre une chaîne "s"
provoquant un débordement
au sein de l'espace des
variables locales

Ecrasement de l'adresse de retour

0x0000

Execution Stack

SP →

SP+4 →

Function
Local Variables

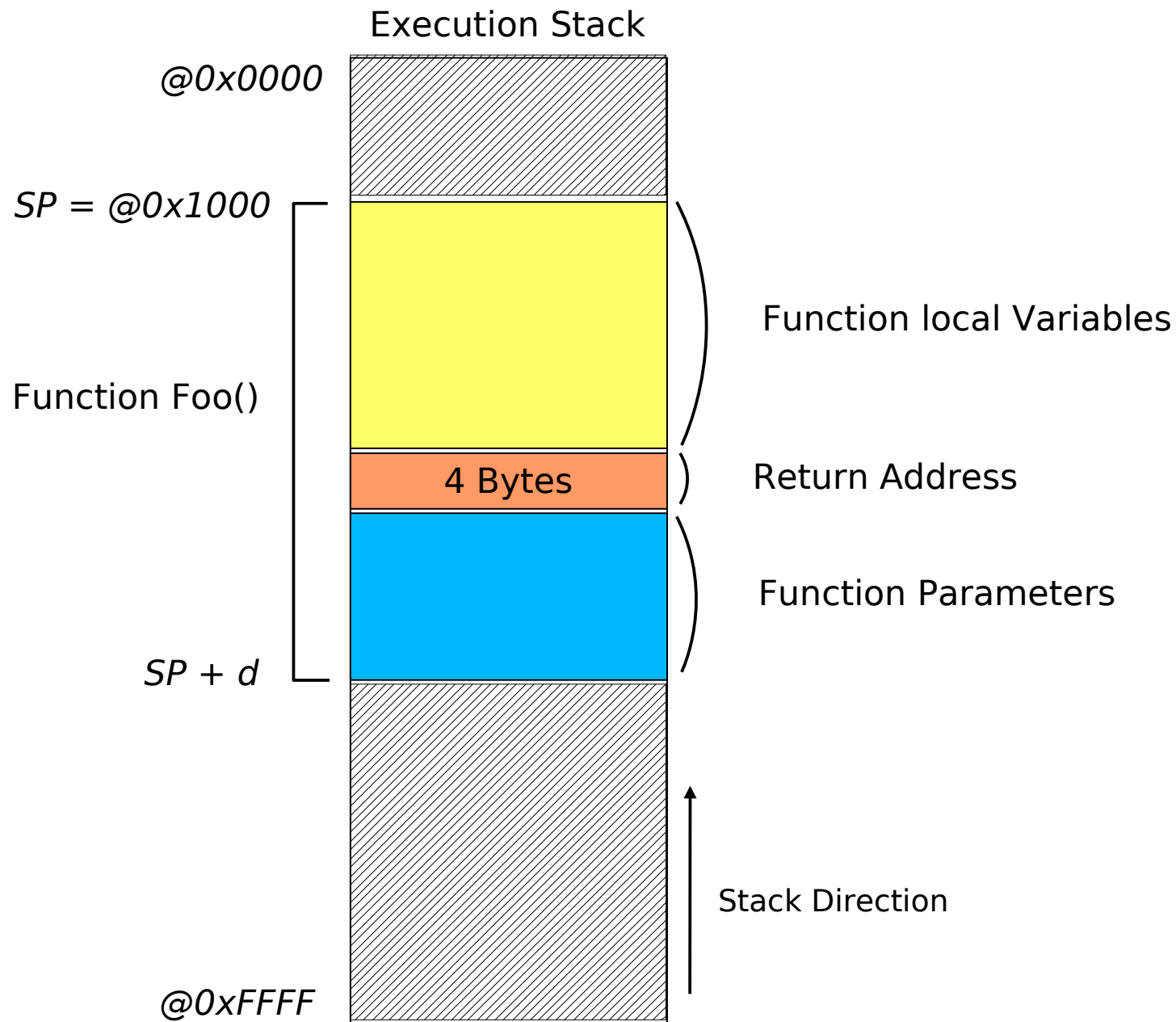
Return Address

Function
Parameters

0xffff

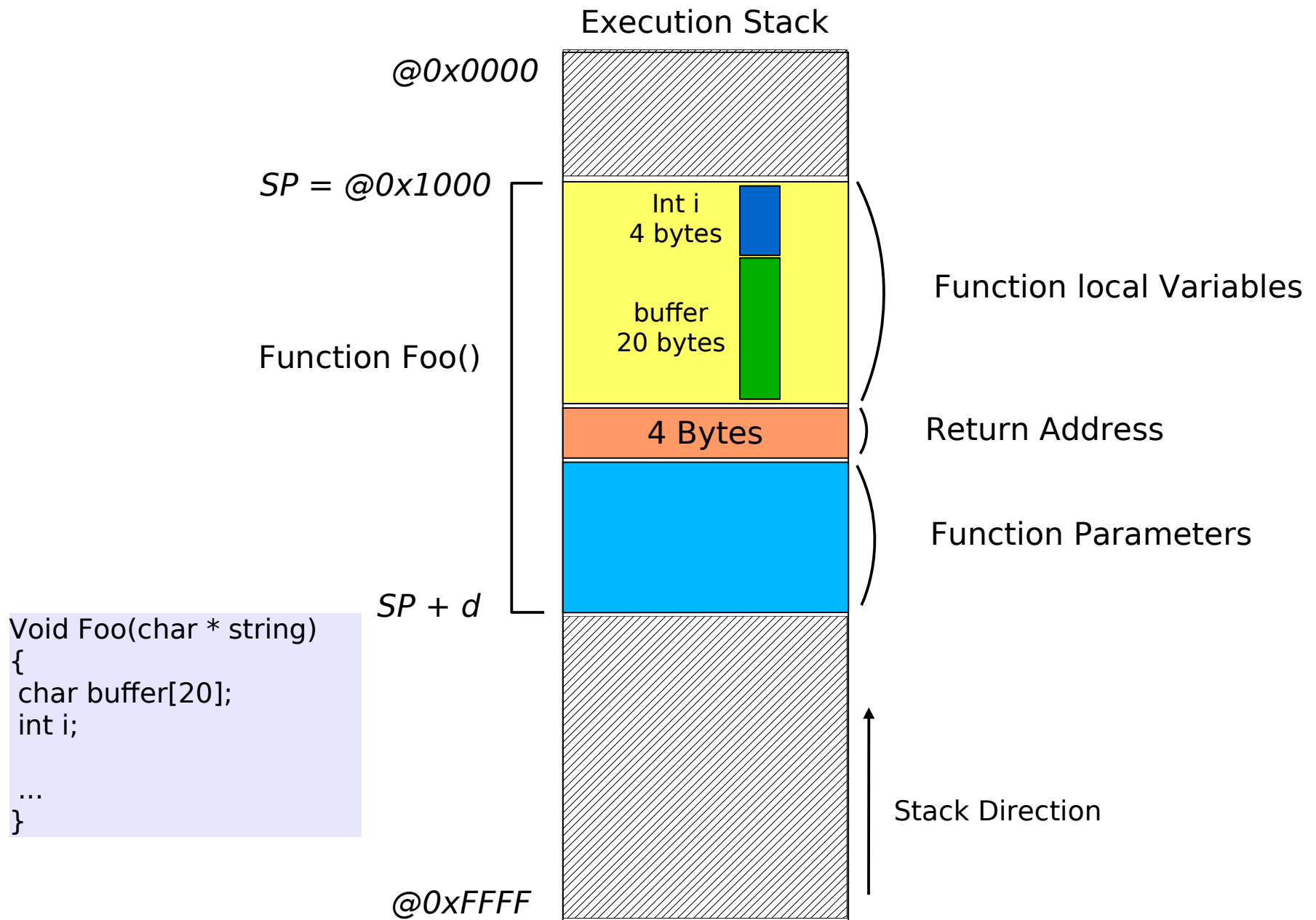


Attaques par débordement de Pile : déroulement sur x86

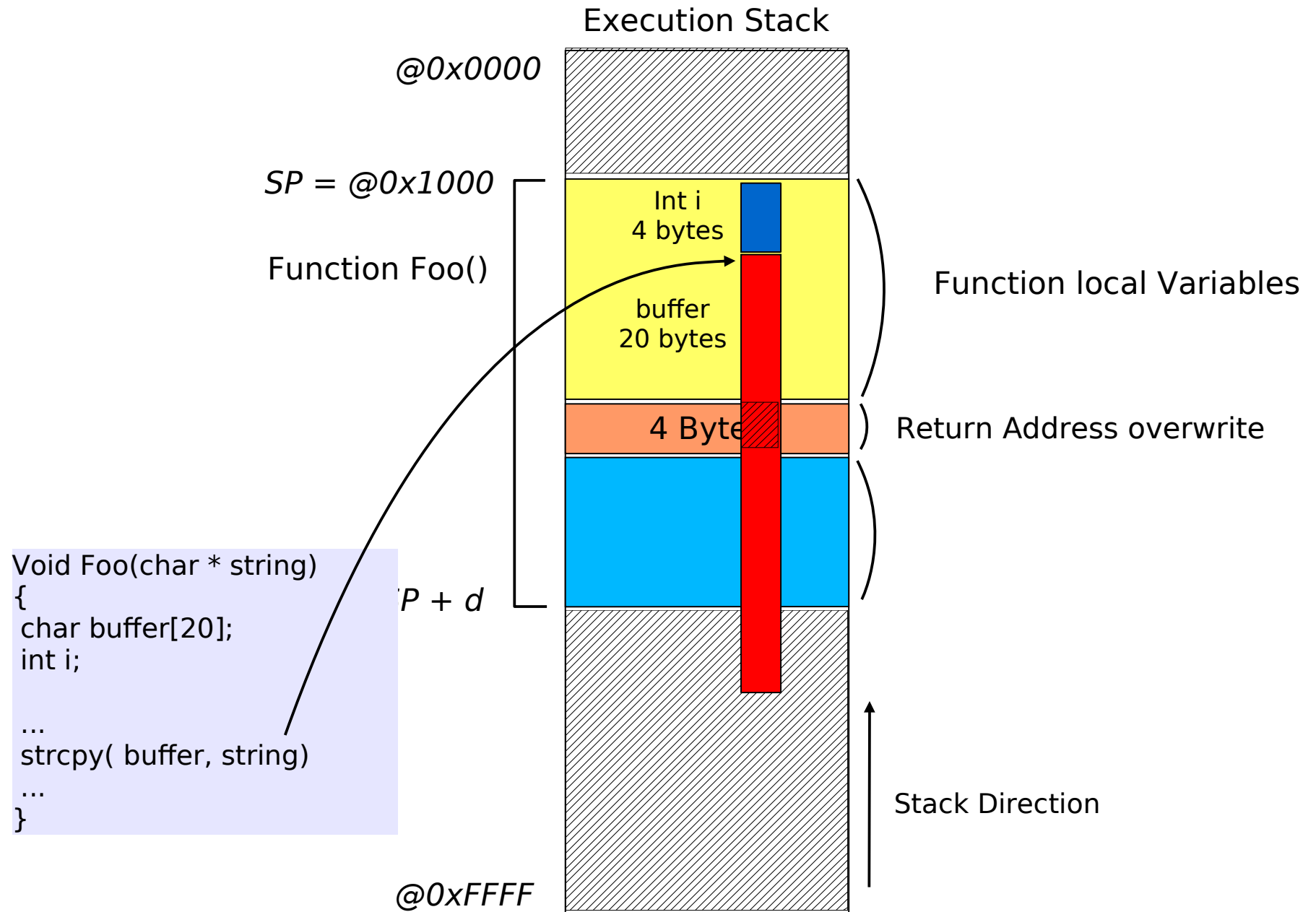




Attaques par débordement de Pile : déroulement sur x86



Attaques par débordement de Pile : déroulement sur x86





Attaques par débordement de Pile : déroulement sur x86

Créer un chaîne "string"
(~**200 o.**) telle que :

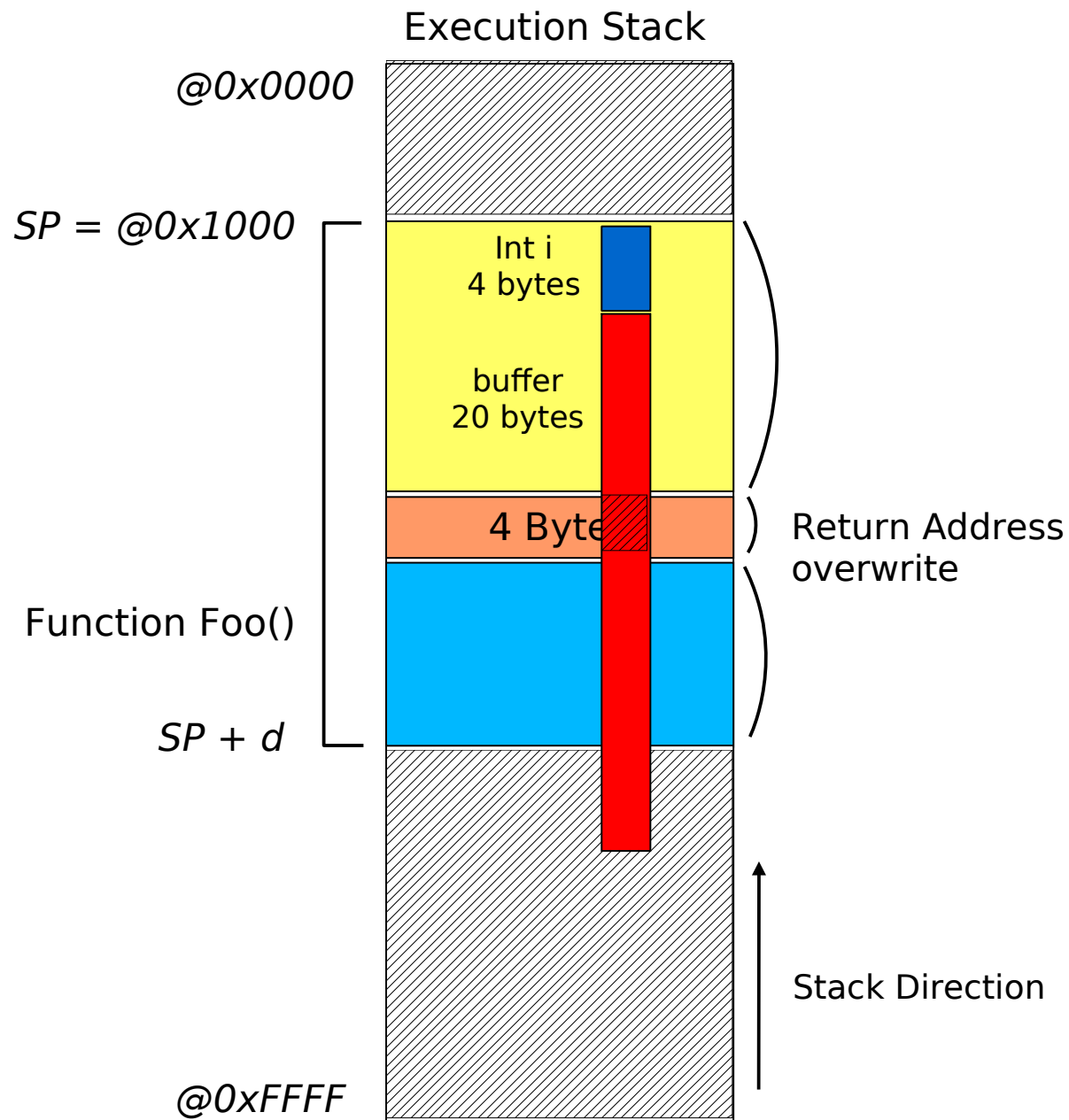
- a) L'adresse de retour pointe sur un code forgé par nous
- b) Contienne un code à exécuter

La première difficulté est de déterminer où est stockée l'adresse de retour.

C'est beaucoup plus facile si on a accès au code source.

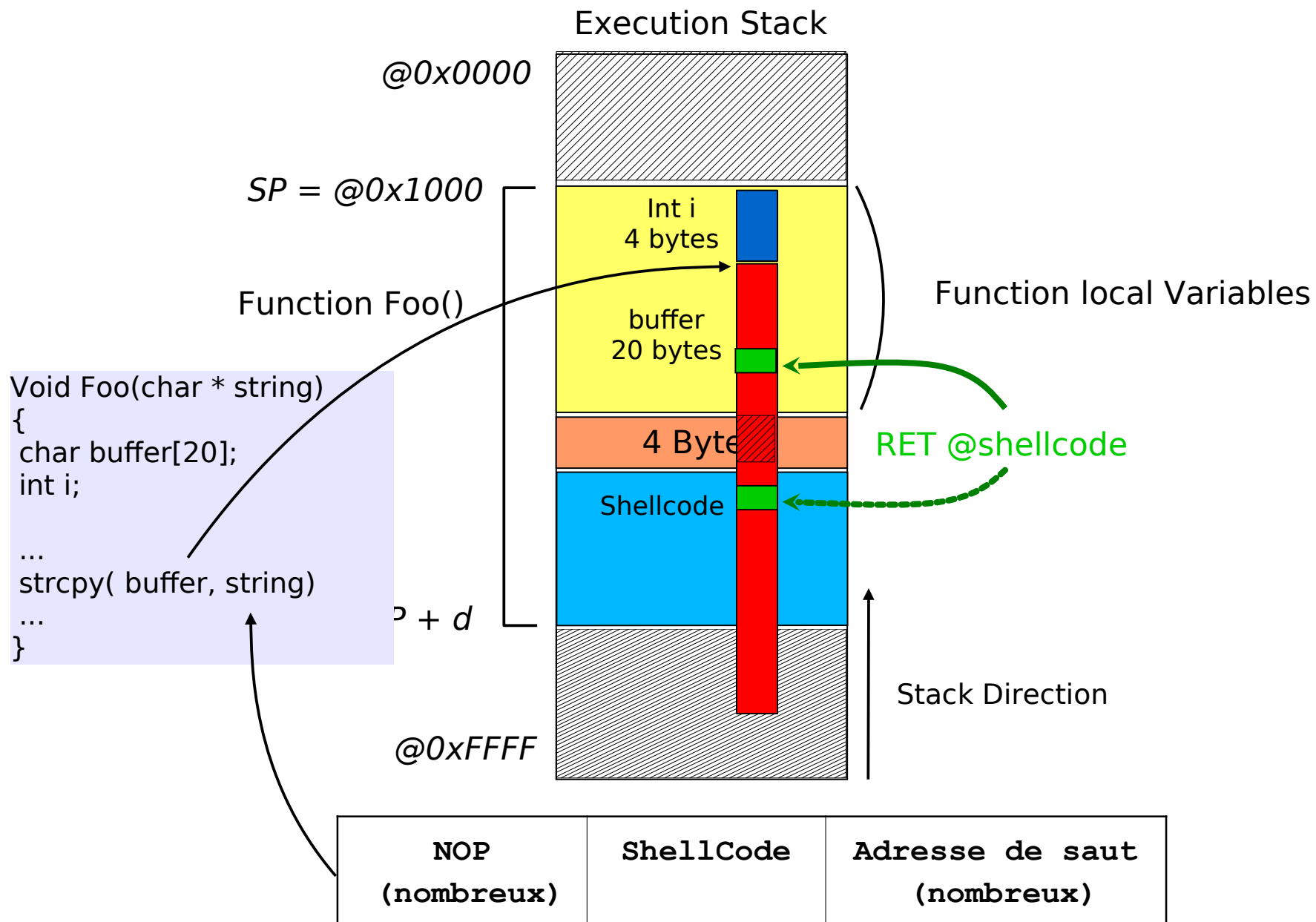
La deuxième difficulté est de déterminer l'adresse où sera stocké le code injecté.

On peut se donner de la marge en remplissant d'instructions sans effet (NOP).



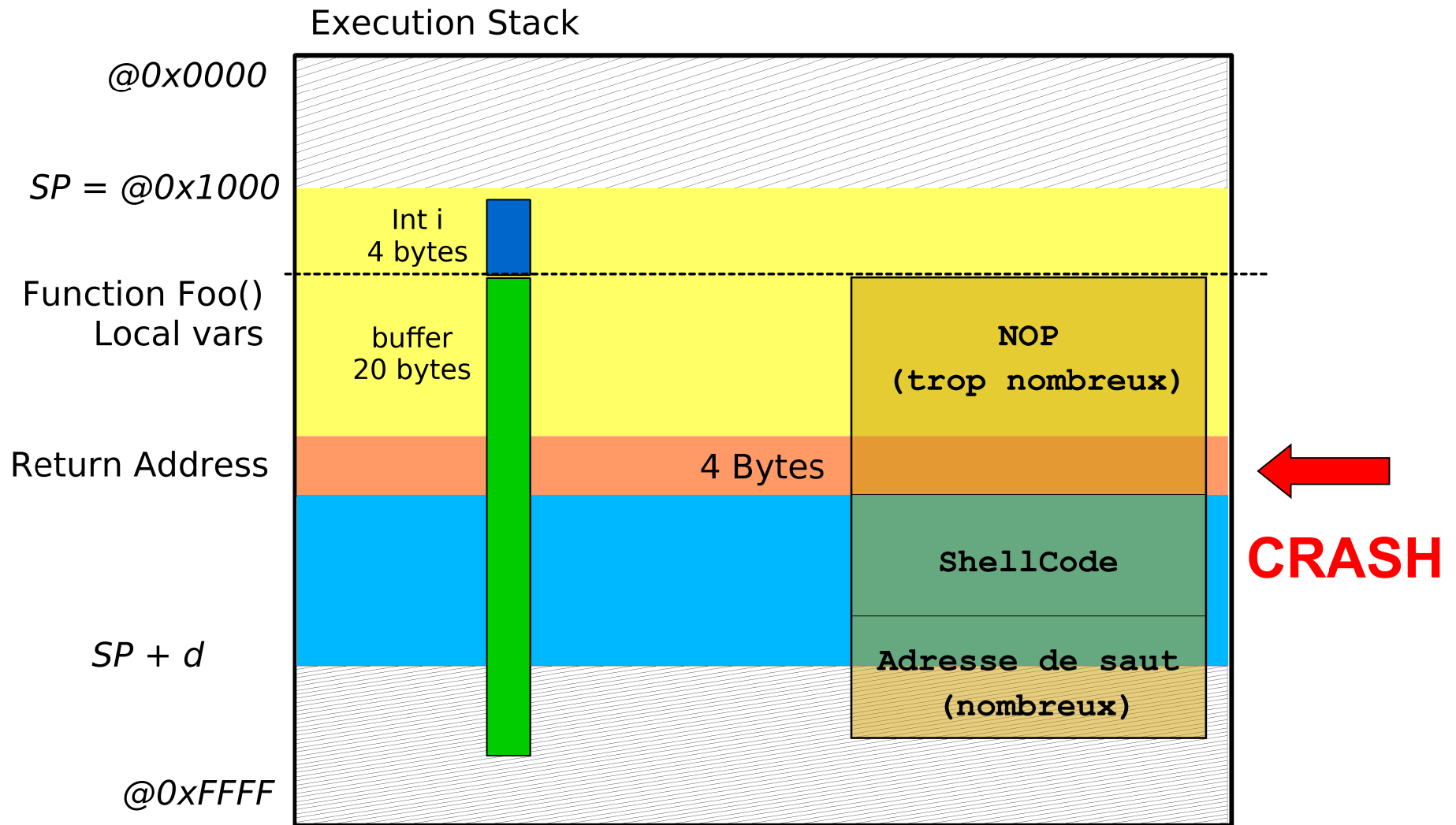


Attaques par débordement de Pile : déroulement sur x86



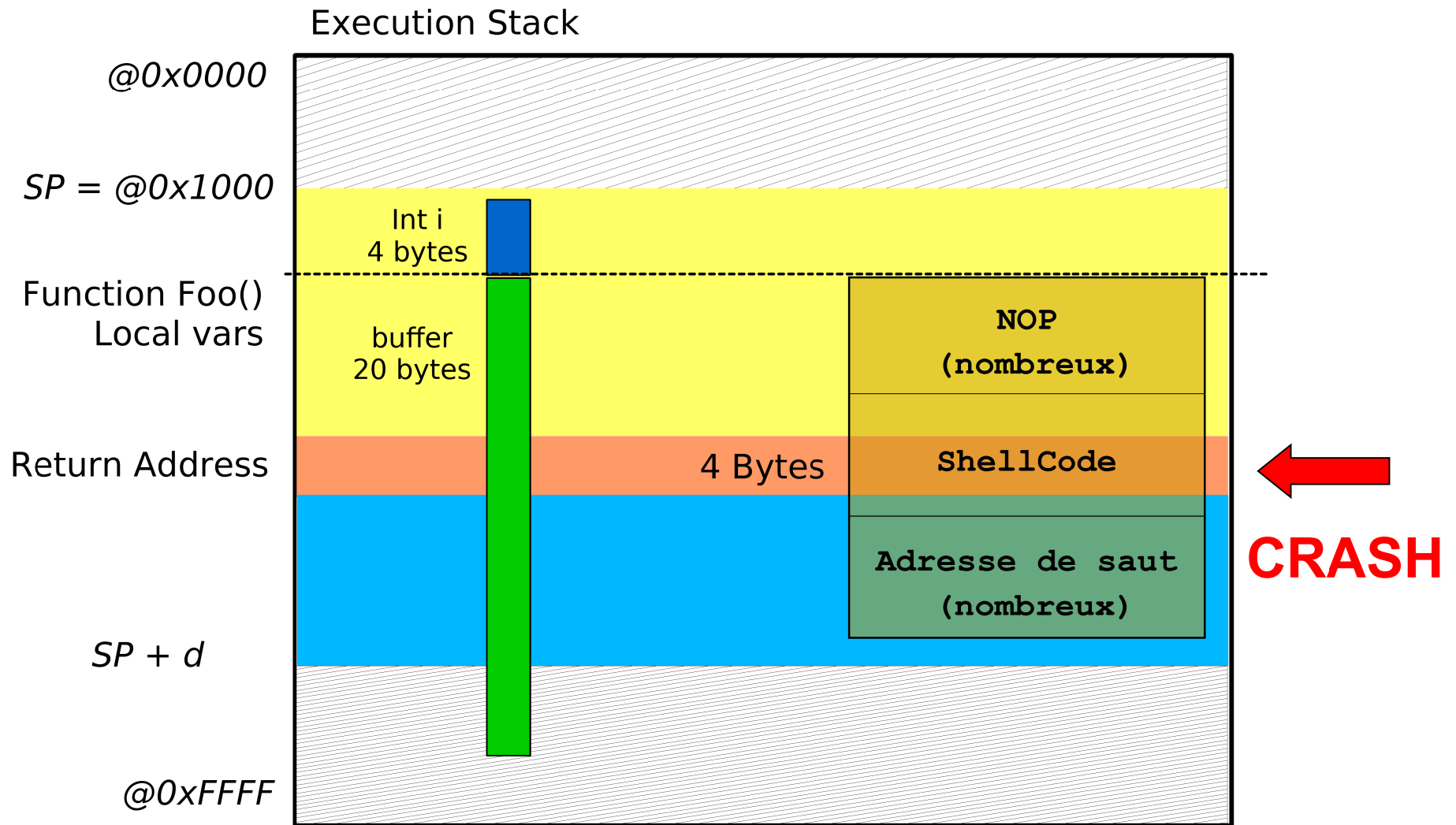


Attaques par débordement de Pile : déroulement sur x86



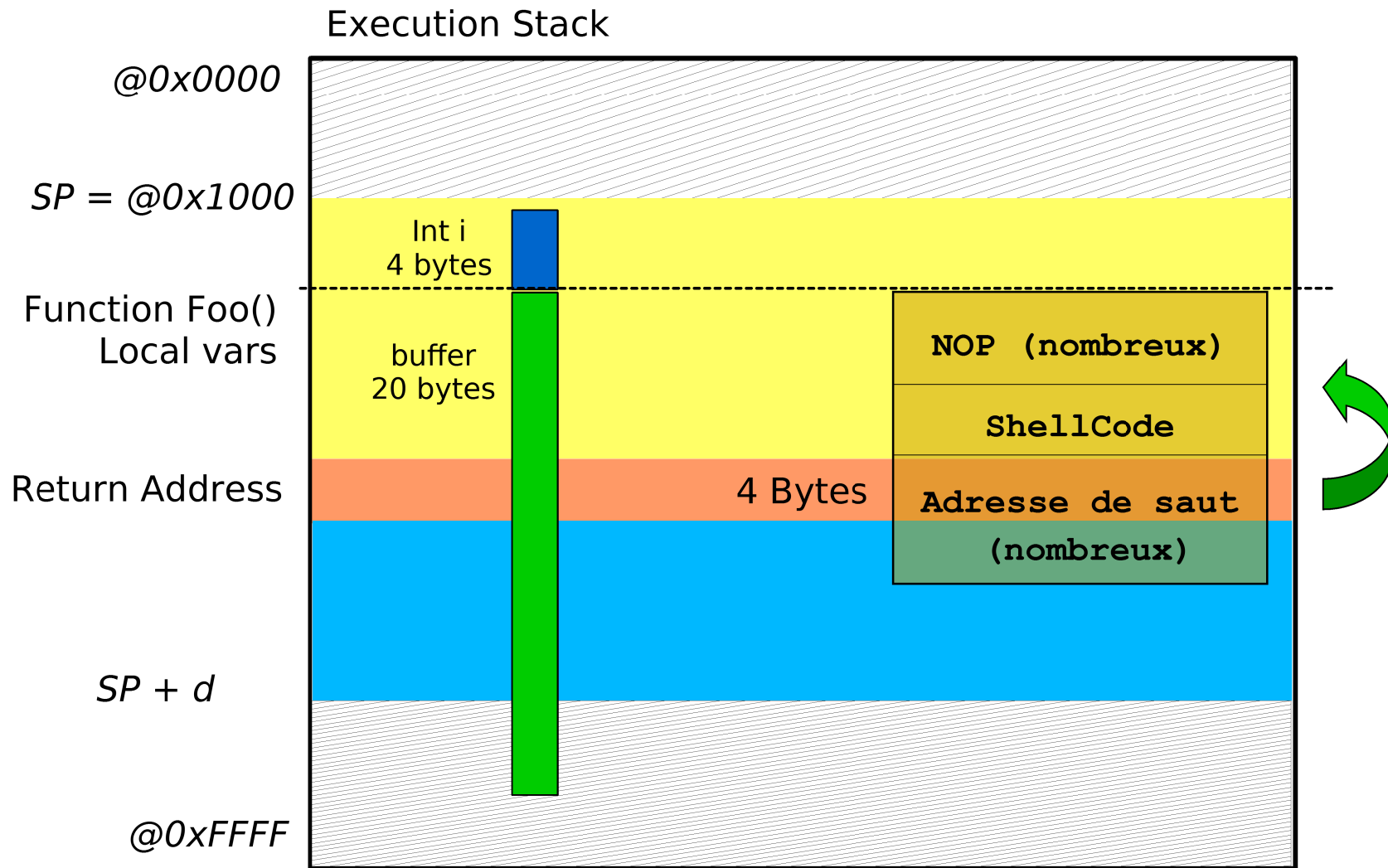


Attaques par débordement de Pile : déroulement sur x86





Attaques par débordement de Pile : déroulement sur x86





Attaques par débordement de Pile : le ShellCode

- On désigne sous le terme de shellcode un code permettant la création d'un shell.
 - En unix/linux, il s'agit de récupérer le code assembleur généré par l'appel à une commande exec sur un shell (sh/bash/csh) et exit (en cas d'échec)

- On écrit le code C

```
#include <stdio.h>
void main() {
    char *name[2]; name[0] = "/bin/sh"; name[1] = NULL;
    execve(name[0], name, NULL);
}
```

- On obtient le code assembleur par

```
gcc -o shellcode -ggdb -static shellcode.c
gdb shellcode
```

Ou gcc -S -o example1.s example1.c

- Le code ne doit contenir aucune adresse absolue (JUMP absolu)
- Le shellcode sera chargé dans l'application par une chaîne spécialement forgée
 - On doit éliminer les caractères 0x00 (NULL) dans le code assembleur !!!
 - Il existe des shellcodes qui résistent au toupper() et tolower()
- Cela fonctionne car la pile (stack) est en lecture/écriture



Attaques par débordement de Pile : exemple de ShellCode

```

System Call          xor %eax,%eax  # remplace mov %eax, $00
setuid()             xor %ebx, %ebx
                    xor %ecx,%ecx
                    mov $0x17, %eax  # interruption setuid(0, 0)
                    int $0x80
                    xor %eax,%eax
                    xor %edx,%edx
                    push %ebx        #sauver ebx
                    push $0x68732f6e
                    push $0x69622f2f
                    mov %esp, %ebx
                    lea (%esp, 1), %edx # LoadEffAdr. "/bin/sh"
                    push %eax        # sauver eax, ebx
                    push %ebx
                    lea (%esp, 1), %ecx
System Call          mov $0xb, %eax  #interruption syst execve
execve()            int $0x80
                    xor %eax, %eax  # en cas d'échec
                    mov $0x1, %eax  # interruption syst exit()
                    int $0x80

```

```
int execve(const char *filename, char *const argv [], char *const envp[]);
```



Attaques par débordement sur le TAS

De plus en plus de développeur d'OS fournissent des patches pour les bofs

- Empêche l'exécution de code dans la pile
- ➔ développement des « Heap Overflow » (« hof »)

C'est un « overflow » qui s'attaque aux données allouées dynamiquement (TAS)

Le TAS sert à stocker les données dynamiques manipulées

- Bloc de données « malloc »
- Les images d'un viewer, les fichiers MP3 d'un lecteur de musique

Un Hof est plus complexe à mettre en place qu'un bof

- Nécessite une connaissance approfondie du système
 - Comment sont allouer les blocs de mémoire ?

Ce type d'attaque est essentiellement basée sur la fabrication de fichiers de données malformés

- L'attaquant doit provoquer le traitement du fichier
- L'attaquant doit attendre le traitement du fichier

Exemple

- Faille GDI+, Faille WMF, Faille WMV



Attaques par débordement sur le TAS

Les données du TAS peuvent contenir des pointeurs

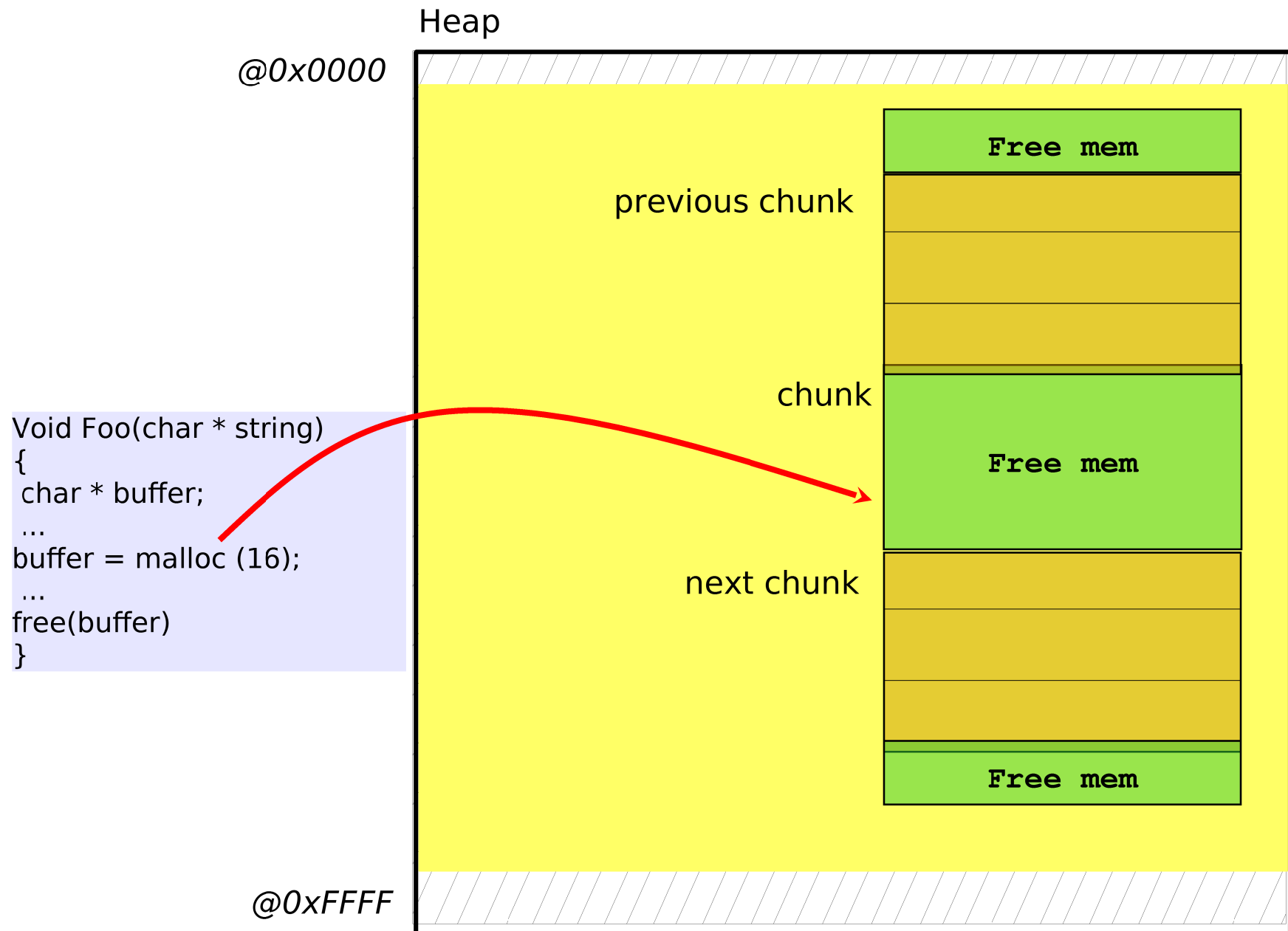
- Ils peuvent être corrompus
- Les données peuvent être corrompues

Exemple :

- Corruption d'une structure utilisée pour un setuid
- Corruption d'une chaîne de nom de fichier
- Corruption d'un pointeur de fonction
 - Pour pointer sur une fonction libc
 - Pour pointer sur un ShellCode contenu sur le tas, la pile

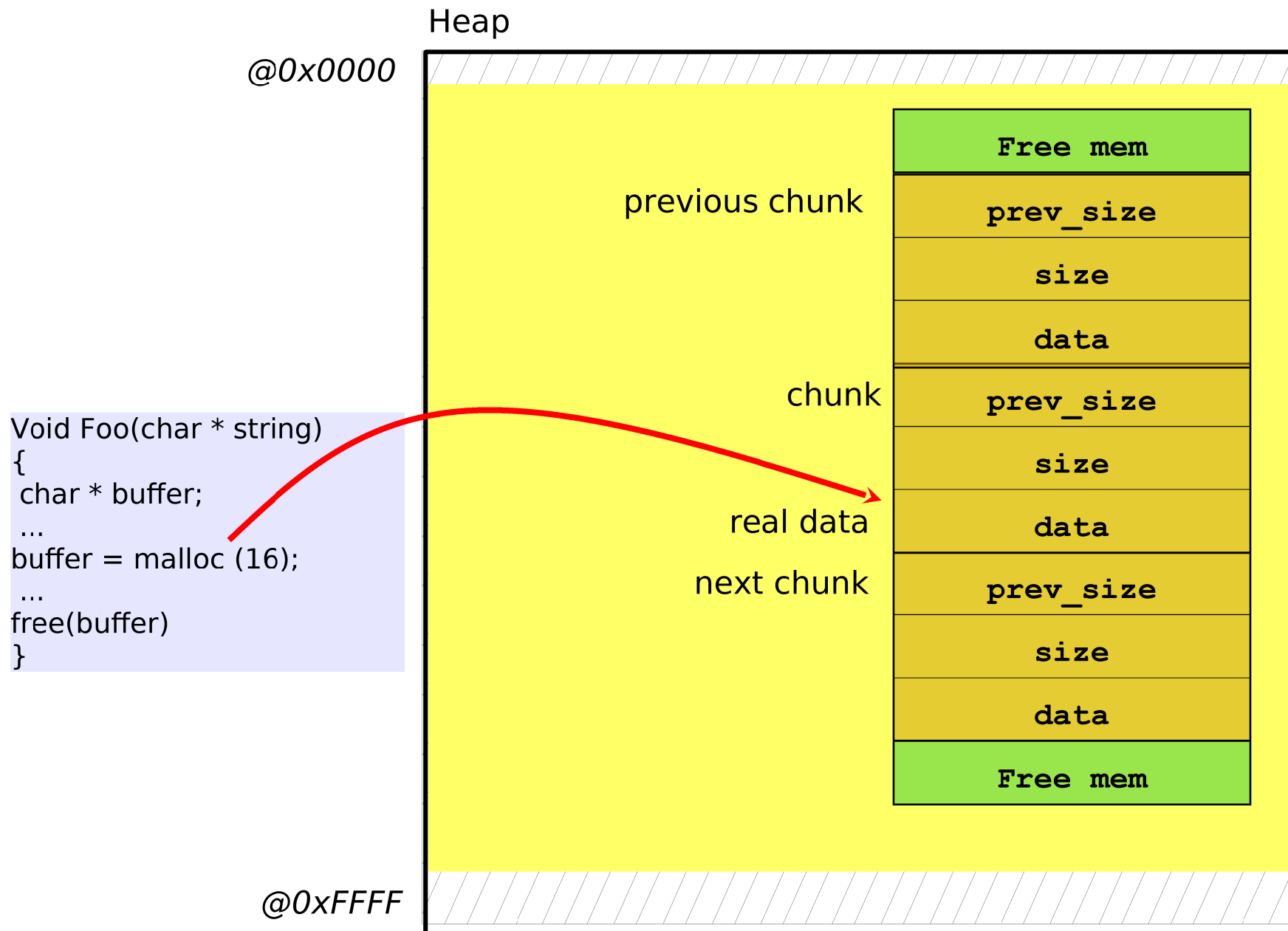


Attaques par débordement sur le TAS : GNU C



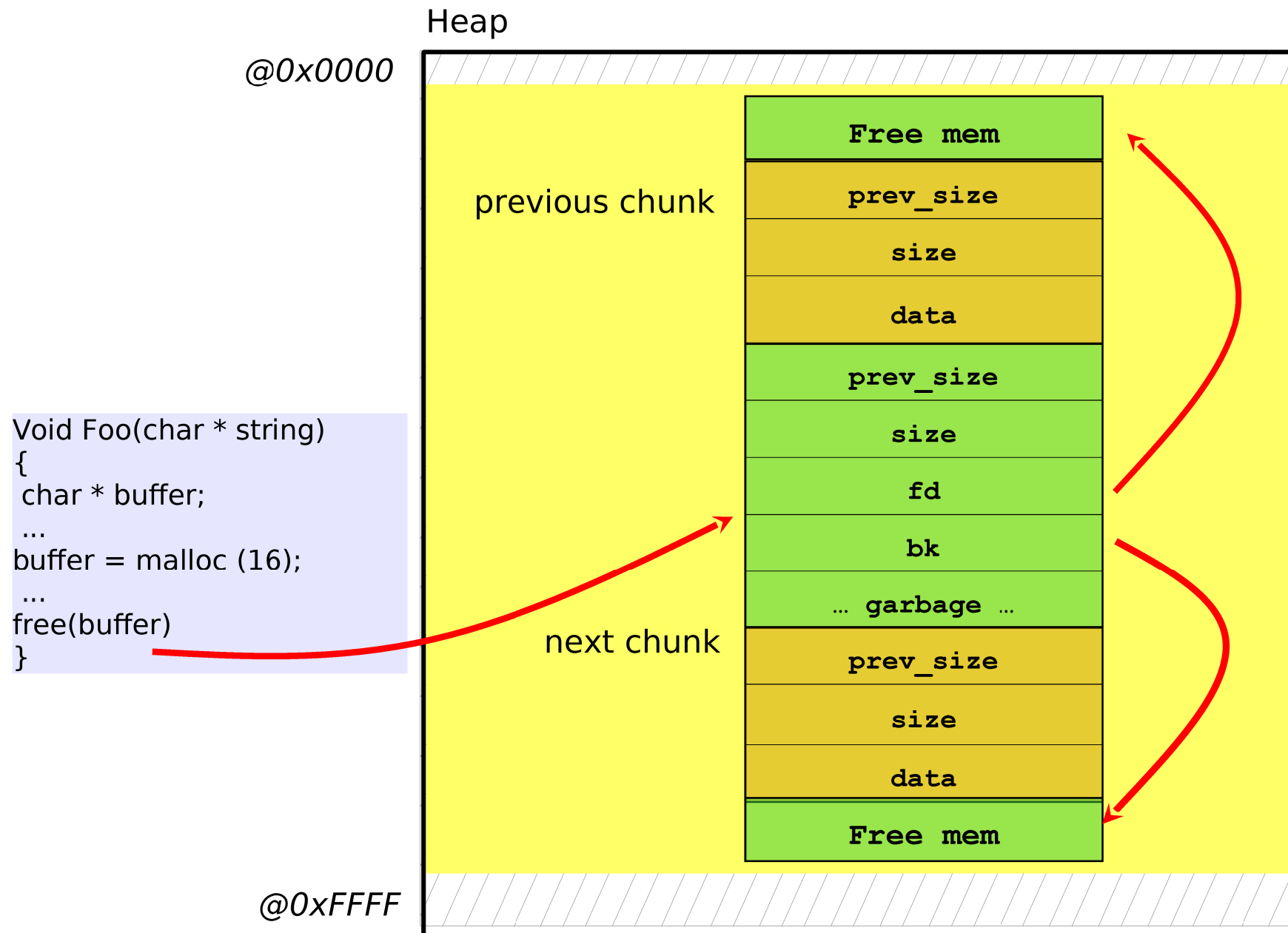


Attaques par débordement sur le TAS : GNU C





Attaques par débordement sur le TAS : GNU C





Attaques par débordement sur le TAS : GNU C

But:

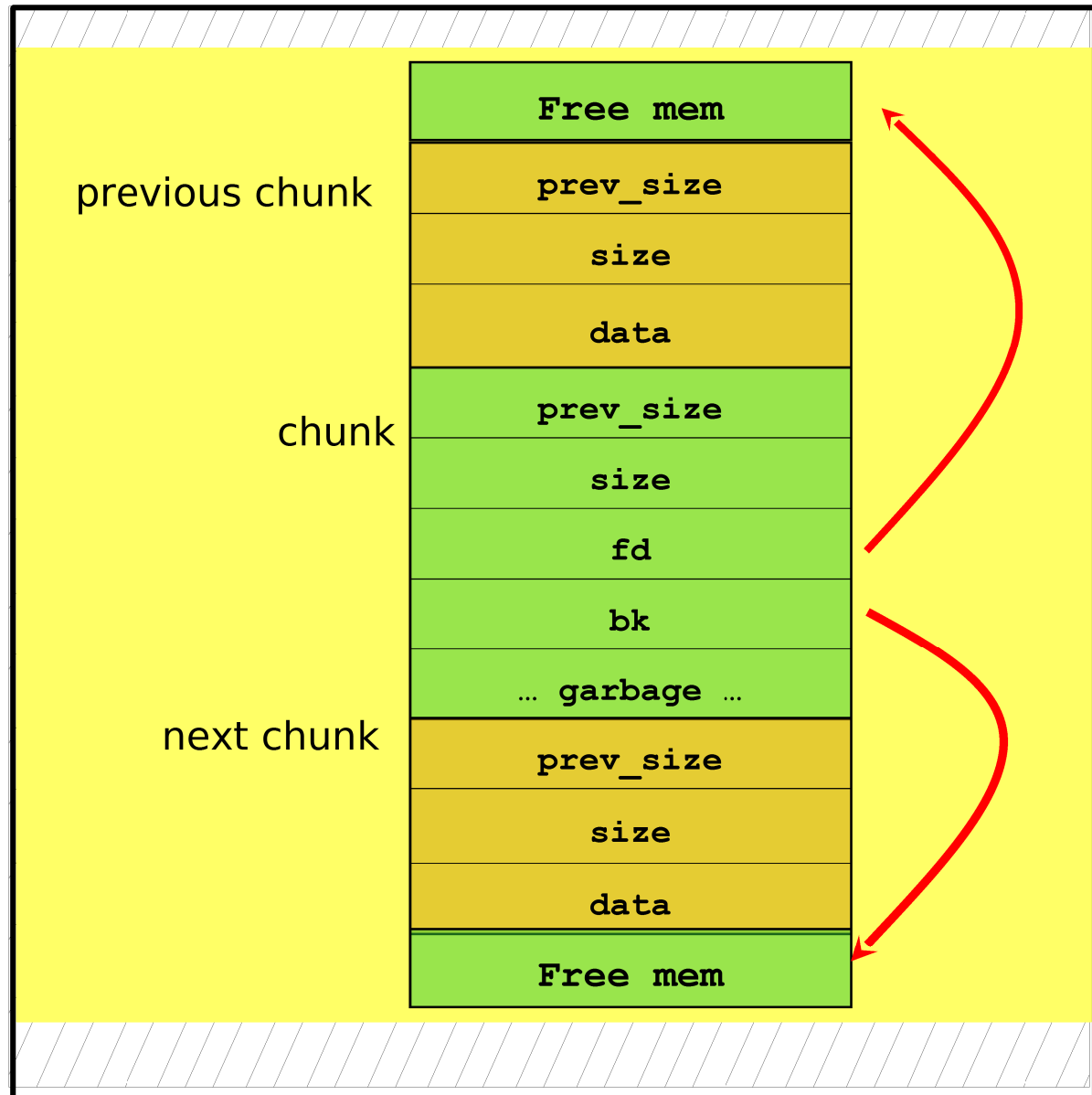
- Corrompre « fd » et « bk »
- Corrompre un autre bloc de données

Moyen

- Créer un buffer overflow

@0x0000

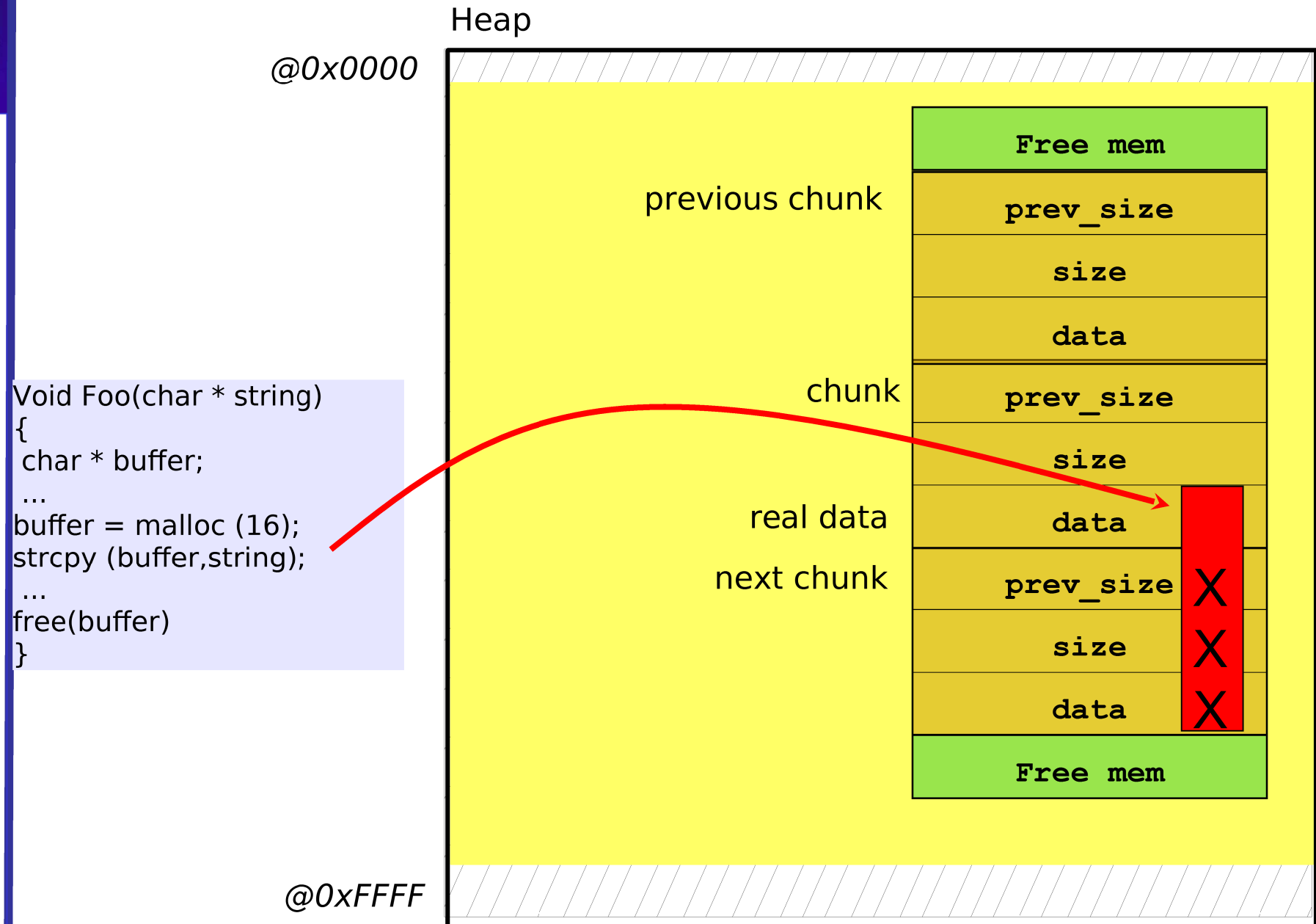
Heap



@0xFFFF

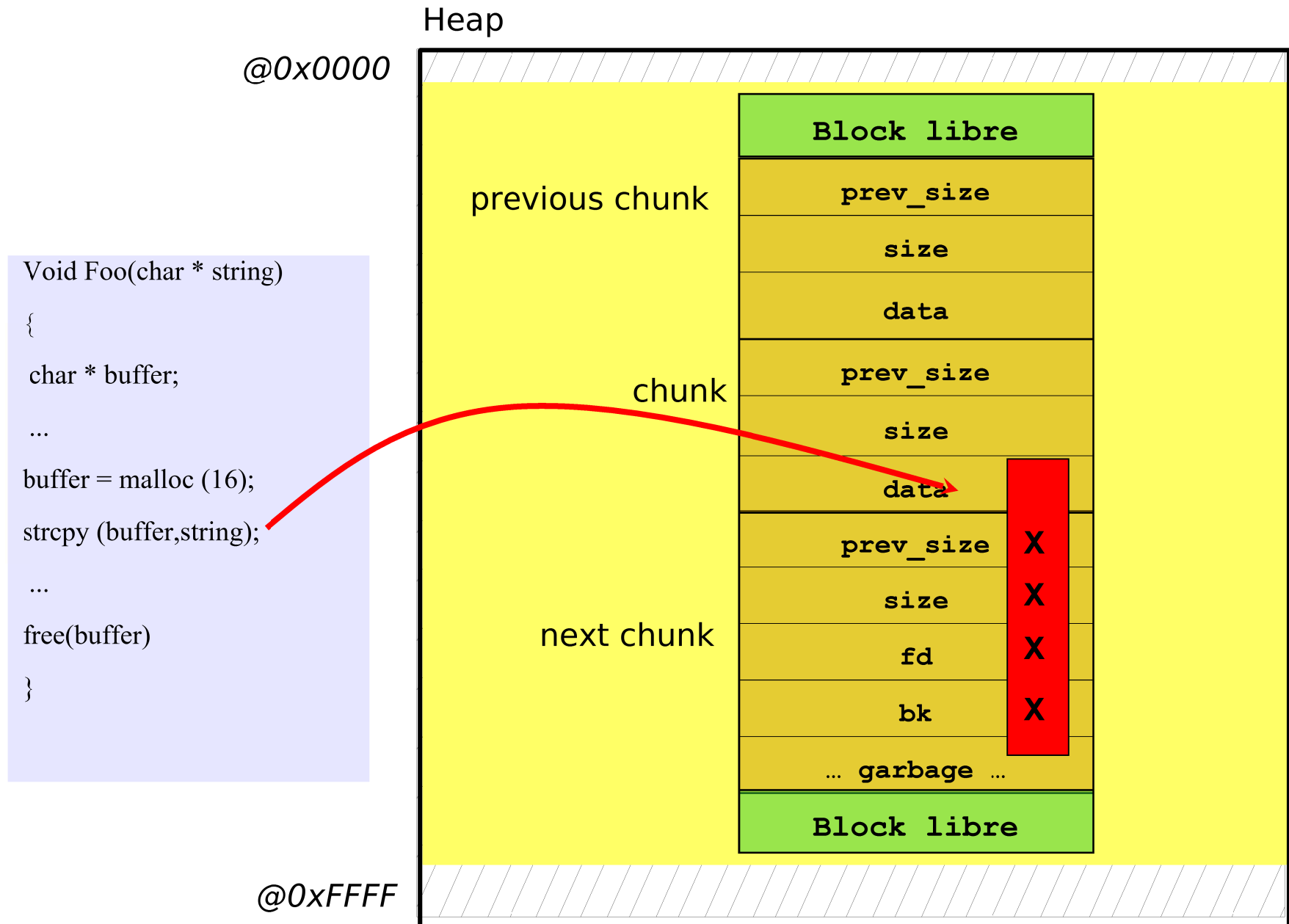


Attaques par débordement sur le TAS : GNU C





Attaques par débordement sur le TAS : GNU C





Attaques par débordement sur un espace réduit

En cas d'espace local trop faible pour stocker le code

- Il est possible de stocker le code ailleurs en mémoire
 - En particulier dans l'environnement du processus
 - Le processus hérite de l'environnement
- De faire pointer l'adresse de retour sur ce code

Il existe de nombreuses variantes

- $\exists$ des variantes moins détectables par les IDS !
 - Voir les codes polymorphes (auto-mutable)

Le ShellCode peut aussi servir à (quelques idées en l'air):

- Copier /bin/sh en /tmp/monshell et faire un suid
- Ouvrir un shell connecté à une socket réseau



Attaques par corruption de « chaîne de texte »

« String Format attack »

Caractéristiques

- Type d'attaque découverte en 2000 (récent)
- Utilise la pile ou le tas qui sont R/W
- Reproduire l'effet des « bof » (dépassement)
- Exploitation des formats de sprintf, printf, et consorts
 - En particulier utilisation du format « %n »

Il s'agit la aussi de mauvaises habitudes de programmation

- Méthode incorrecte: `printf(string);`
 - Un exemple très simple d'instruction dangereuse
 - Si `string = « %s%s%s%s%s »`, 99% de chance de crash
- Méthode correcte: `printf("%s", string);`



Attaques par corruption de « chaîne de texte »

Il existe des exemples plus complexes

```
{  
char outbuf[512];  
char buffer[512];  
sprintf (buffer, "ERR Wrong command: %400s", user);  
.....  
sprintf (outbuf, buffer);  
}
```

Code apparemment anodin

- user="%497d\x3c\xd3\xff\xbf<nops><shellcode>"
- Lors du premier sprintf → copie normale dans buffer
- Lors du second sprintf → le contenu de buffer est interprété
 - %497d → prend la première valeur de la pile et génère une chaîne
 - La taille : 497 caractères + longueur(« ERR Wrong command : »)
 - C'est supérieur à 512 → dépassement de capacité avec le formatage de la chaîne

Le but ultime → reproduire un « bof » avec le formatage printf

- Cf. exemple précédent



Attaques par fichier interposé

Exploitation de droits mal définis sur des fichiers

Peut être couplée avec une attaque de type « race condition »

Le meilleur exemple : « /tmp »

- Un grand nombre de programme suid y stocke des données temporaires
- N'importe qui peut y écrire
- On peut créer des liens vers des fichiers « root »

Le but

- utiliser une application ayant les droits root pour modifier les fichiers root via des liens fabriqués
- Si l'application ne vérifie pas correctement la nature du fichier
→ problème

Exemple (désuet) :

- mktemp crée des fichiers temporaires unique mais pas imprédictibles
- Faire un lien de /tmp/fichier_temporaire vers /.rhosts
- Si l'application ajoute une ligne « ++ » au fichier, c'est gagné
- Après il ne reste plus qu'à faire « rsh localhost -l root »



Attaques par « course de vitesse »

Principe :

- Système multitâches (de nombreux processus)
- Profiter du système à un instant où il est vulnérable
 - Exploit moins difficile à réaliser que l'attaque par débordement
 - « Il suffit » de réunir les conditions pour que cela arrive
- Changer les conditions d'exécutions pendant l'exécution d'un programme
 - Manipuler les flux de données, insérer des données dans le programme
- Mauvaises suppositions sur des liens de causalité
 - Ex sur les fichiers → vérification des droits, ouverture, lecture
- Comme d'habitude → les programmes « suid » sont les cibles !

But :

- Obtenir les droits « root »
- Accéder aux informations manipulées par le service
- Empêcher le fonctionnement correct du service
- Bloquer le travail des autres utilisateurs



Attaques par « course de vitesse »

TOCTTOU = « Time Of Check To Time Of Use »

- Exploitation de la latence une mesure et son utilisation
 - Applicable aux fichiers (**en particulier aux fichiers sur /tmp**)
 - Les scripts de par leur lenteur (langage interprété) y sont très sensibles
- Obtention des informations par logiciels simples et analyse des logs produits
 - strace (appels systèmes), ltrace (appels dynamiques), nm (liste des symboles externes)



Attaques par « course de vitesse » : exemple « passwd »

- Erreur sur HP/UX et SunOS, avec « passwd »
 - Programme Suid puisque doit modifier /etc/passwd
- Déroulement normal des opérations
 1. Ouverture et lecture du fichier « /etc/passwd »
➔ obtention de l'entrée utilisateur
 2. Création et ouverture d'un fichier « ptmp » dans le répertoire du fichier « passwd »
 3. Ouvrir à nouveau le fichier « /etc/passwd » et copier le contenu dans « ptmp » (en mettant à jour l'utilisateur).
 4. Fermer le fichier « passwd », « ptmp »
 5. Renommer (pour remplacer) le fichier « passwd » par « ptmp »
- Le programme « passwd » peut travailler sur un fichier spécifié



Attaques par « course de vitesse » : exemple « passwd »

Créer un fichier ayant un format compréhensible par plusieurs services (« passwd » et « rlogin »)

localhost account ::::

Cette ligne est valide pour les fichiers « rhost » et « passwd »

Exécution de passwd (programme suid) dont on va contourner le fonctionnement normal

Comment ?

- En s'insérant entre les manipulations de « passwd »



Attaques par « course de vitesse » : exemple « passwd »

- A. On utilise un répertoire pointé par un lien symbolique.
Lien `~attaquant/link` → `~attaquant/tmprep`
Création du fichier « `~attaquant/link/.rhosts` » avec « `localhost account :::::` »
- 1. Ouverture et lecture du fichier « `~attaquant/link/.rhosts` »
→ obtention de l'entrée utilisateur
- A. L'attaquant change le lien symbolique : `~attaquant/link` → `~cible`
- 2. Création et ouverture d'un fichier « `~attaquant/link/ptmp` » dans le répertoire du fichier passé en paramètre « `~attaquant/link` »
- A. « `passwd` » est root, il crée le fichier « `ptmp` » chez la cible
MAJ du lien symbolique : `~attaquant/link` → `~attaquant/tmprep`
- 3. Ouvrir à nouveau le fichier « `~attaquant/link/.rhosts` » et copier le contenu dans « `ptmp` » (en mettant à jour l'utilisateur).
- A. La ligne « `localhost account :::::` » est copiée
MAJ du lien symbolique : `~attaquant/link` → `~cible`
- 4. Fermer le fichier « `passwd` », « `ptmp` »
- A. La fermeture n'utilise pas le lien symbolique mais travaille sur le descripteur
- 5. Renommer (pour remplacer) le fichier « `~attaquant/link/.rhosts` » par le fichier « `ptmp` »
- A. La copie remplace le fichier « `.rhost` » du répertoire cible par le `ptmp`

Attaques par « course de vitesse » : exemple « tmpwatch »

« tmpwatch »

- Utilisé dans beaucoup de distribution linux
- Purge régulièrement les /tmp des fichiers
- Souffre du bugs de « courses de vitesse »
Sur les anciennes versions

Course à l'effacement

- « tmpwatch » → lstat, vérifie la date, unlink
- Système multitâche → interruption possible entre « lstat » et « unlink »

1^{ère} Attaque possible

- On enlève le fichier après le « lstat » et on crée un lien avant le « unlink »

2^{ème} Attaque possible

- Les fichiers « mktemp » sont uniques mais pas imprédictibles
- « tmpwatch » peut être maintenu en état d'exécution (remplissage de tmp)
- On se débrouille pour créer un fichier tmp qui sera utilisé par un programme cible
- Les cibles privilégiées sont : browser web, mailer, ...
- On utilise « tmpwatch » pour effacer le fichier crée et le remplacer par un autre

Un exemple

- « logrotate » stocke ses scripts post/pre-rotate dans des fichier « /tmp/logrotate.XXXXXXX »



Attaques par « course de vitesse » : Solution au TOCTOU

Eviter l'utilisation de fichier autant que possible (surtout dans les scripts shell)

- Le bit « s » est interdit sur les scripts → **TROP DANGEREUX !**

Lors de la création d'un fichier

- utiliser O_EXCL|O_CREAT (**O_EXCL ne fonctionne pas sur NFS v1 et v2**)
- Appliquer des droits MINIMA avec open (**pas après la création**)
- Ils ne doivent pas être mis dans un endroit manipulable par un attaquant
 - Eviter autant que possible les répertoires partagés (/tmp)
- Pour les fichiers temporaires → **utiliser mkstemp()**
 - « mktemp » ne fixe pas les droits
 - « tempnam » utilise la variable d'environnement TMP



Attaques par « course de vitesse » : Solutions au TOCTOU

Changer les droits du processus

- Apprendre à utiliser `setuid/getuid/setreuid`

Utiliser les fonctions `fchown`, `fstat`, `fchmod`

- Elles travaillent sur les handles que VOUS avez ouverts
- Eviter `chown`, `lstat`, ... qui travaillent sur des noms
- Ne pas utiliser « `access` » pour vérifier les droits

Faites attention aux manipulations du FS pendant un parcours en récursion

Utilisation des verrous POSIX (« `fcntl` »)

Méthode de verrouillage obligatoire

- Pour chaque lecture/écriture, on vérifie le verrou
- Problème en cas de crash du service